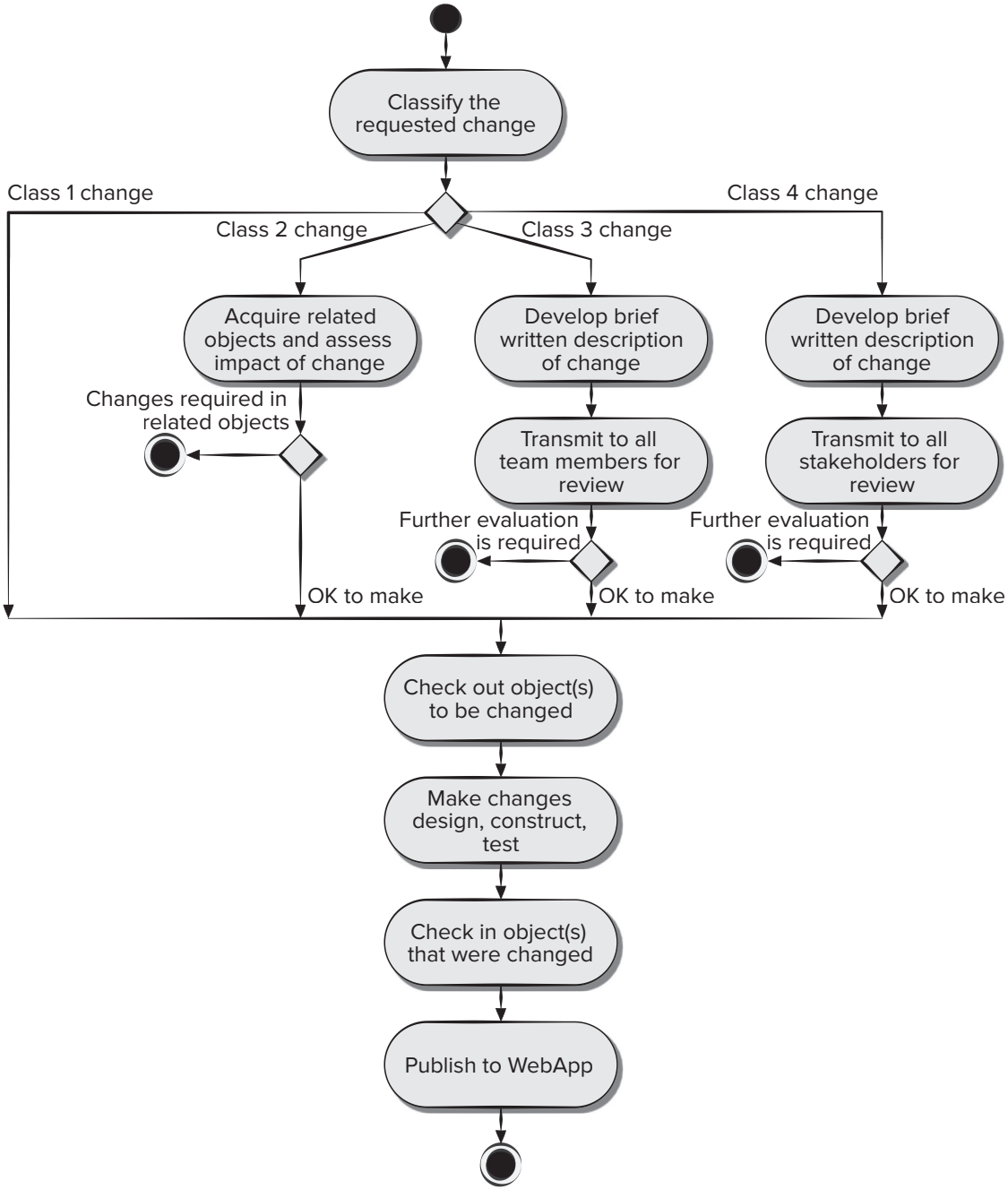
FIGURE 22.7 Managing changes for WebApps



Referring to the figure, class 1 and 2 changes are treated informally and are handled in an agile manner. For a class 1 change, you would evaluate the impact of the change, but no external review or documentation is required. As the change is made, standard check-in and check-out procedures are enforced by configuration repository tools. For class 2 changes, you should review the impact of the change on related objects (or ask other developers responsible for those objects to do so). If the change can be made without requiring significant changes to other objects, modification occurs without additional review or documentation. If substantive changes are required, further evaluation and planning are necessary.

Class 3 and 4 changes are also treated in an agile manner, but some descriptive documentation and more formal review procedures are required. A *change description*—describing the change and providing a brief assessment of the impact of the change—is developed for class 3 changes. The description is distributed to all members of the team who review it to better assess its impact. A change description is also developed for class 4 changes, but in this case all stakeholders conduct the review.

22.6.2 Content Management

Content management is related to configuration management in the sense that a content management system (CMS) establishes a process (supported by appropriate tools) that acquires existing content (from a broad array of app and/or game configuration objects), structures it in a way that enables it to be presented to an end user, and then provides it to the client-side environment for display.

The most common use of a content management system occurs when a dynamic application is built. Apps and games create screen displays "on the fly." That is, the user typically performs an action that the software responds to by changing the information displayed on the screen. The user action may cause the app to query a server-side database; it then formats the information accordingly and presents it to the user.

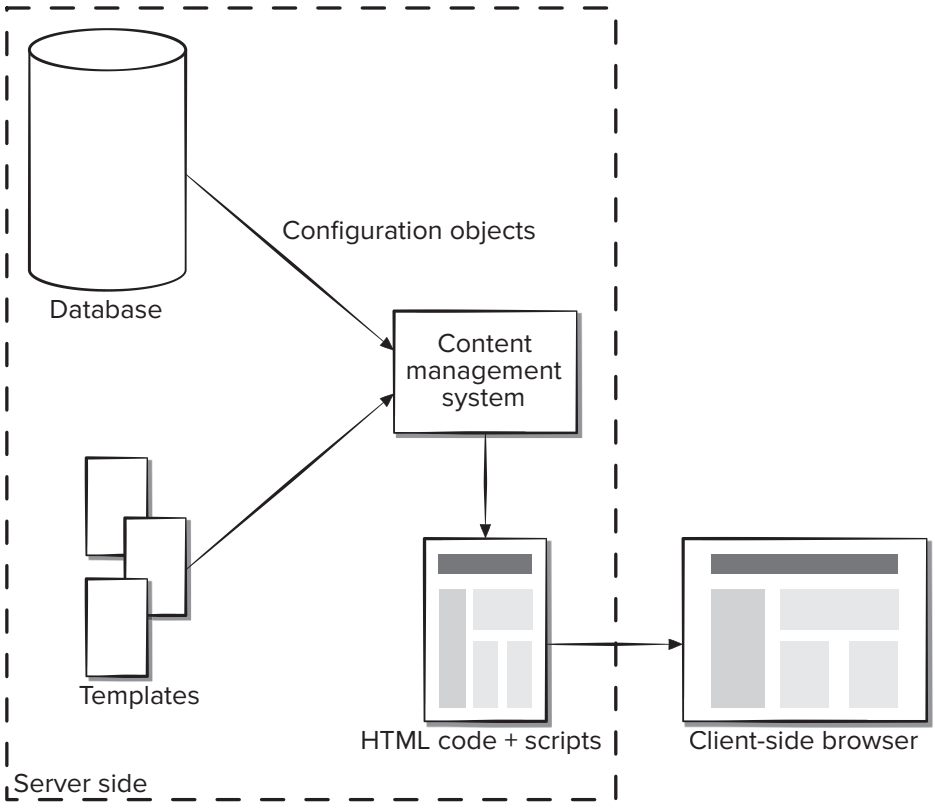
For example, a music store (e.g., Apple iTunes) provides hundreds of thousands of tracks for sale. When a user requests a music track, a database is queried and a variety of information about the artist, the CD (e.g., its cover image or graphics), the musical content, and sample audio are all downloaded and configured into a standard content template. The resultant page is built on the server side and passed to the client side for examination by the end user. A generic representation for WebApps is shown in Figure 22.8.

22.6.3 Integration and Publishing

Content management systems are useful for composing Web services to create context-aware MobileApps and updating game-level scenes at run time, as well as building dynamic Web pages. In the most general sense, a CMS "configures" content for the end user by invoking three integrated subsystems: a collection subsystem, a management subsystem, and a publishing subsystem [Boi04].

The Collection Subsystem. Content is derived from data and information that must be created or acquired by a content developer. The *collection subsystem* encompasses all actions required to create and/or acquire content, and the technical functions that are necessary to (1) convert content into a form that can be represented by a mark-up

FIGURE 22.8
Content management system



language (e.g., HTML, XML), and (2) organize content into screens that can be displayed efficiently on the client side.

Content creation and acquisition (often called *authoring* or *level design* for games) commonly occurs in parallel with other development activities and is often conducted by nontechnical content developers. This activity combines elements of creativity and research and is supported by tools that enable the content author to characterize content in a manner that can be standardized for use within the app or game.

Once content exists, it must be converted to conform to the requirements of a CMS. This implies stripping raw content of any unnecessary information (e.g., redundant graphical representations), formatting the content to conform to the requirements of the CMS, and mapping the results into an information structure that will enable it to be managed and published.

The Management Subsystem. Once content exists, it must be stored in a repository, cataloged for subsequent acquisition and use, and labeled to define (1) current status (e.g., is the content object complete or in development?), (2) the appropriate version of the content object, and (3) related content objects. Configuration management is

performed within this subsystem. Therefore, the *management subsystem* implements a repository that encompasses the following elements:

- **Content database.** The information structure that has been established to store all content objects.
- **Database capabilities.** Functions that enable the CMS to search for specific content objects (or categories of objects), store and retrieve objects, and manage the file structure that has been established for the content.
- **Configuration management functions.** The functional elements and associated workflow that support content object identification, version control, change management, change auditing, and reporting.

In addition to these elements, the management subsystem implements an administration function that encompasses the metadata and rules that control the overall structure of the content and the manner in which it is supported.

The Publishing Subsystem. Content must be extracted from the repository, converted to a form that is amenable to publication and formatted so that it can be transmitted to client-side screen displays. The publishing subsystem accomplishes these tasks using a series of templates. Each *template* is a function that builds a publication using one of three different components [Boi04]:

- **Static elements.** Text, graphics, media, and scripts that require no further processing are transmitted directly to the client side.
- **Publication services.** Function calls to specific retrieval and formatting services that personalize content (using predefined rules), perform data conversion, and build appropriate navigation links.
- **External services.** Access to external corporate information infrastructure such as enterprise data or "backroom" applications.

A content management system that encompasses each of these subsystems is applicable for major Web and mobile projects. However, the basic philosophy and functionality associated with a CMS are applicable to all dynamic applications.

22.6.4 Version Control

As apps and games evolve through a series of increments, a number of different versions may exist at the same time. One version (the current operational app) is available via the Internet for end users; another version (the next app increment) may be in the final stages of testing prior to deployment; a third version is in development and represents a major update in content, interface aesthetics, and functionality. Configuration objects must be clearly defined so that each can be associated with the appropriate version. Without some type of control, developers and content creators may end up overwriting each other's changes.

It's likely that you've experienced a similar situation. To avoid it, a version control process is required.

1. **A central repository for the app or game project should be established.** The repository will hold current versions of all configuration objects (content, functional components, and others).

2. **Each developer creates his own working folder.** The folder contains those objects that are being created or changed at any given time.
3. **The clocks on all developer workstations should be synchronized.** This is done to avoid overwriting conflicts when two developers make updates that are very close to one another in time.
4. **As new configuration objects are developed or existing objects are changed, they are imported into the central repository.** The version control tool will manage all check-in and check-out functions from the working folders of each developer. The tool should also provide automatic e-mail updates to all interested parties when changes to the repository are made.
5. **As objects are imported or exported from the repository, an automatic, time-stamped log message is made.** This provides useful information for auditing and can become part of an effective reporting scheme.

The version control tool maintains different versions of the app and can revert to an older version if required.

22.6.5 Auditing and Reporting

In the interest of agility, the auditing and reporting functions are deemphasized during the development of games or apps.[9] However, they are not eliminated altogether. All objects that are checked into or out of the repository are recorded in a log that can be reviewed at any point in time. A complete log report can be created so that all members of the team have a chronology of changes over a defined period of time. In addition, an automated e-mail notification (addressed to those developers and stakeholders who have interest) can be sent every time an object is checked in or out of the repository.

22.7 SUMMARY

Software configuration management is an umbrella activity that is applied throughout the software process. SCM identifies, controls, audits, and reports modifications that invariably occur while software is being developed and after it has been released to a customer. All work products created as part of software engineering become part of a software configuration. The configuration is organized in a manner that enables orderly control of change.

The software configuration is composed of a set of interrelated objects, also called software configuration items, that are produced as a result of some software engineering activity. In addition to software engineering work products, the development environment that is used to create software can also be placed under configuration control. All SCIs are stored within a repository that implements a set of mechanisms and data

9 This is beginning to change. There is an increasing emphasis on SCM as one element of application security [Fug14]. By providing a mechanism for tracking and reporting every change made to every application object, a change management tool can provide valuable protection against malicious changes.

structures to ensure data integrity, provide integration support for other software tools, support information sharing among all members of the software team, and implement functions in support of version and change control.

Once a configuration object has been developed and reviewed, it becomes a baseline. Changes to a baselined object result in the creation of a new version of that object. The evolution of a program can be tracked by examining the revision history of all configuration objects. Version control is the set of procedures and tools for managing the use of these objects.

Change control is a procedural activity that ensures quality and consistency as changes are made to a configuration object. The change control process begins with a change request, leads to a decision to make or reject the request for change, and culminates with a controlled update of the SCI that is to be changed.

The configuration audit is an SQA activity that helps to ensure that quality is maintained as changes are made. Status reporting provides information about each change to those with a need to know.

Configuration management for apps or games is similar in most respects to SCM for conventional software. However, each of the core SCM tasks should be streamlined to make it as lean as possible, and special provisions for content management must be implemented.

PROBLEMS AND POINTS TO PONDER

22.1. Why is the first law of system engineering true? Provide specific examples for each of the four fundamental reasons for change.

22.2. What are the four elements that exist when an effective SCM system is implemented? Discuss each briefly.

22.3. Assume that you're the manager of a small project. What baselines would you define for the project, and how would you control them?

22.4. Design a project database (repository) system that would enable a software engineer to store, cross-reference, trace, update, change, and so forth all important software configuration items. How would the database handle different versions of the same program? Would source code be handled differently than documentation? How will two developers be precluded from making different changes to the same SCI at the same time?

22.5. Research an existing SCM tool, and describe how it implements control for versions, variants, and configuration objects in general.

22.6. Research an existing SCM tool, and describe how it implements the mechanics of version control. Alternatively, read two or three papers on SCM and describe the different data structures and referencing mechanisms that are used for version control.

22.7. Develop a checklist for use during configuration audits.

22.8. What is the difference between an SCM audit and a technical review? Can their function be folded into one review? What are the pros and cons?

22.9. Briefly describe the differences between SCM for conventional software and SCM for WebApps or MobileApps.

22.10. Describe the value of continuous integration tools to agile software developers.

Design element: Quick Look icon magnifying glass: © Roger Pressman